Classic approach: PPO with Preferences instead of Rewards

Before preference-oriented methods were introduced

- PPO was the main method for RL with preferences
- It still is

But PPO is based on rewards (scalars) and we only have preferences (rankings).

So we first have to translate preferences into rewards

- and then use PPO as usual

The Reward Model

It is difficult and manually-intensive for a human to translate preferences to rewards

- Need to be consistent across examples
- No guiding principles

Example

```
Task 1: reward scale 0-100
Task 2: reward scale 0-10
```

Are we saying that the perfect example of Task 1 is 10 times more important than that of Task 2 ?

- or is the human using a different scale ?

Instead, we train a *Reward Model* r_ϕ

- Neural Network parameterized by ϕ
- to translate rankings to rewards

The model must satisfy

$$\mathrm{r}_\phi(x, y^+) > \mathrm{r}_\phi(x, y^-)$$

for all preferences

$$(x, y^+, y^-)$$

That is: the reward of the preferred choice is higher than the less preferred.

The per-example Loss function is even stricter than this condition

$$\loss^{\ip}_{\text{reward}}(\phi, x, y^+, y^-) = -\log \sigma\big(\mathrm{r}_\phi(x, y^+) - \mathrm{r}_\phi(x, y^-)\big)$$

It tries to *maximize the spread* between the two choices.

In the Discussion: we will provide an interesting interpretation of this condition.

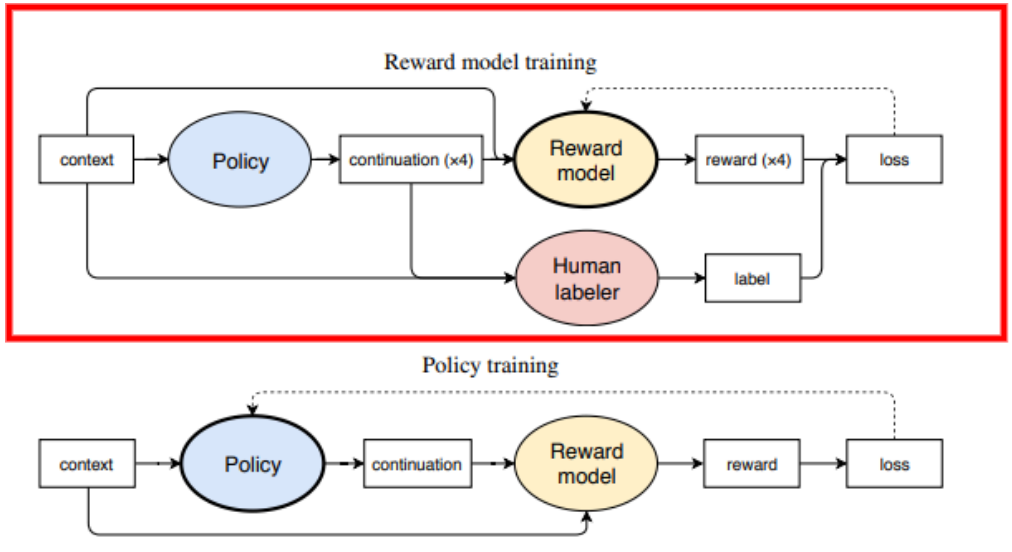
Where does the training data for the Reward model come from ?

- human preferences
    - costly
- LLM "Judge" preferences
    - synthetic data in practice

The training causes the reward model to imitate the preferences inherent in the training data.

Reward model: training

- A prompt (context) is fed to the both a human (offline) and the model
- The model creates model responses (continuation)
- The Reward Model and the Human both rank the responses (calculate a reward)
- The Loss function penalizes the model for model rewards that deviate from the human reward



context = prompt; continuation = response

Source: https://arxiv.org/pdf/1909.08593.pdf#page=2

$\loss^{\ip}_{\reward}(\phi, x, y^+, y^-)$ interpretation

Consider the form of the per-example Loss

$$\loss^{\ip}_{\reward}(\phi, x, y^+, y^-) = -\log \sigma\big(\mathrm{r}_\phi(x, y^+) - \mathrm{r}_\phi(x, y^-)\big)$$

It is similar to the Binary Cross-Entropy Loss term for Positive examples

$$\log \pr\x$$

where

$$\pr\x = \sigma(\z)$$

That is, in Logistic Regression

- we compute a score z
- convert $\mathrm{\backslash z}$ to a probability $\mathrm{\backslash pr\backslash x} = \sigma(\mathrm{\backslash z})$
    - probability of "example being Positive" via the sigmoid function
- use Binary Cross Entropy as the Loss
    - sum of terms for
        - Positive examples: $-\log(\sigma(z))$
        - Negative examples: $\log(1-\sigma(z))$

In our case: the triples are all "Positive" examples so cross entropy collapses to $-\log(\sigma(z))$

So we can view

$$\loss^{\ip}_{\text{reward}}(\phi, x, y^+, y^-) = -\log \sigma\big(\mathrm{r}_\phi(x, y^+) - \mathrm{r}_\phi(x, y^-)\big)$$

as the Cross Entropy Loss of predicting the probability

$$\prc y^+ \text{ preferred to } y^- x$$

where the score z is

$$z = \mathrm{r}_\phi(x, y^+) - \mathrm{r}_\phi(x, y^-)$$

Discussion

The usual critique of PPO is that involves multiple models

- usual instances of the same model for Policy

They are

- Policy Model
- Reference Model
    - recall: PPO Surrogate Loss constrains Policy Model updates to not deviate too far from the Reference Model
- Value/Critic: for advantage computation

PPO for Preferences involves an additional model

- Reward Model

This has motivated the search for alternate methods for Preference Data.

Model Type	Typical Size	Role
Policy Model	Large-scale pretrained LM (billions)	Generate responses
Reward Model	Smaller/fine-tuned LM or distilled (hundreds of millions)	Score outputs based on preferences

Direct Preference Optimization (DPO)

The key idea of PPO was the translation of preferences into rewards.

We did this by training a Reward model to maximize the

- *spread in rewards*
- between preferred y^+ and non-preferred y^-

Direct Preference Optimization (DPO) takes this core idea but simplifies the process

It directly maximizes the

- *spread in probability*
- between preferred y^+ and non-preferred y^-
- being generated by the policy π

No need to convert preferences to rewards !

This method is

- Supervised Fine Tuning
- rather than Reinforcement Learning

and can be formulated as a Binary Classification problem

- maximize the spread in probability of the Preferred ("Positive" label) vs Non-Preferred ("Negative" label)

Direct Preference Optimization (DPO) is thus a policy optimization method where supervision

- comes from *Preference Data*
  $$(x, y^+, y^-)$$
- rather than rewards

Relation to the Unified Gradient Formulation

DPO also uses a surrogate loss to guide the derivation of the policy.

$$L_{\mathrm{DPO}}(\theta) = -\log \sigma \left(\log \frac{\pi_\theta(y^+|x)}{\pi_\theta(y^-|x)} \right) = -\log \sigma \left(\Delta \right)$$

where

- $\pi_\theta(y^+|x)$
- $\pi_\theta(y^-|x)$

are the probabilities of the *trajectories* resulting in outputs y^+ and y^-.

- The ratio
$$\frac{\pi_\theta(y^+|x)}{\pi_\theta(y^-|x)}$$

is the relative probability of the preferred output, compared to the non-preferred output._

- We take the log of the ratio
    - resulting in log-probabilities, as in the Universal Formulation
- The sigmoid σ converts the relative probability to the range $[0, 1]$.

We can compute the gradient of $L_{\mathrm{DPO}}(\theta)$

We first simplify $L_{\mathrm{DPO}}(\theta)$ by defining

$$\Delta = \log \frac{\pi_\theta(y^+|x)}{\pi_\theta(y^-|x)} = \log \pi_\theta(y^+|x) - \log \pi_\theta(y^-|x)$$

Substituting into $L_{\mathrm{DPO}}(\theta)$

$$L_{\mathrm{DPO}}(\theta) = -\log \sigma\left(\Delta\right)$$

The gradient of this simplified $L_{\mathrm{DPO}}(\theta)$ is $$ \begin{array}{l} \nabla_\theta L_{\mathrm{DPO}}(\theta) & = & -(1 - \sigma(\Delta)) \cdot \nabla_\theta \Delta \\ & = & -(1 - \sigma(\Delta)) \cdot \big(\nabla_\theta \log \pi_\theta(y^+|x) \nabla_\theta \log \pi_\theta(y^-|x) \big) \end{array} $$

This follows from basic rules of calculus

The term

$$(1 - \sigma(\Delta))$$

is interpreted as the *Advantage* of y^+ over y^-.

This advantage is small

- when $\sigma(\Delta) \approx 1$
    - i.e., the model is confident: assigning high probability to the preferred output

Conversely, it is large when the model is uncertain.

So the advantage term adjusts the Gradient update step size depending on how far the probability of the preferred output is from 100%.

The gradient can be interpreted as adjusting the policy

- so as to increase the (log) likelihood
- adjusted by the advantage

$L_{\mathrm{DPO}}(\theta)$ interpretation

Consider the form of the per-example Loss

$$L_{\mathrm{DPO}}(\theta) = -\log \sigma \left(\log \frac{\pi_\theta(y^+|x)}{\pi_\theta(y^-|x)} \right) = -\log \sigma \left(\Delta \right)$$

Just as we observed for $L_{\mathrm{PPO}}(\theta)$ in the section on PPO

- It is similar to the Binary Cross-Entropy Loss term for Positive examples

$$\log \backslash \mathrm{pr} \backslash \mathrm{x}$$

where

$$\backslash \mathrm{pr} \backslash \mathrm{x} = \sigma(\Delta)$$

But, for DPO

- Δ is not the score $\z$ (the logit)
- it is the *spread* in log probabilities of the Preferred and Non-Preferred choices

$$\Delta = \log \frac{\pi_\theta(y^+|x)}{\pi_\theta(y^-|x)} = \log \pi_\theta(y^+|x) - \log \pi_\theta(y^-|x)$$

Thus

$$L_{\text{DPO}}(\theta)$$

is equivalent to the Binary Cross Entropy loss for the problem predicting the probability

$$\prc y^+ \text{ preferred to } y^- x$$

DPO vs PPO

DPO avoids the main critique of PPO

- the need for multiple models
- each potentially consuming many parameters

It avoids the artificial creation of rewards and operates directly on probabilities.

Supervised Fine Tuning of a Classifier-like Loss is used in DPO

- rather than Reinforcement Learning as in PPO

Approach	Data Used	Training Steps	Model Copies Required	Key Challenge
PPO + Reward Model	Preference → Scalar rewards	Train reward model, then PPO optimization	Policy, reference, value, reward	Reward modeling complexity, instability
DPO	Preference pairs directly	Single-stage policy gradient optimization	Only policy	Requires careful pairing, but simpler overall

Pseudo code for DPO

```
# DPO training for LLM
for prompt in training_prompts:
    outputs = [llm.generate(prompt) for _ in range(2)]

    # outputs: [output_0, output_1]
    # preference: 0 if output_0 is preferred, 1 if output_1 is preferred
    preferred_idx = compare_outputs(outputs) # Human or synthetic comparison

    logit_pref = llm.score(outputs[preferred_idx], prompt)
    logit_nonpref = llm.score(outputs[1 - preferred_idx], prompt)

    # DPO loss: maximize difference so preferred > non-preferred
    loss = -logsigmoid(logit_pref - logit_nonpref)

    loss.backward()
    optimizer.step()
```

Group Relative Policy Optimization (GRPO)

Group Relative Policy Optimization (GRPO) is an innovative, policy-based method for learning from Preference Data.

It cleverly circumvents several issues common to RL for Preferences

- Scarcity of training examples
- Difficulty in constructing a Reward model and calibrating rewards

Data Generation

Given training dataset D

- traditional RL learns from each questions/answer pair $(q, a) \in D$

So learning is limited by the size of D.

GRPO creates G synthetic examples from q

- prompting a model multiple (G) times with the same question q
    - with non-zero temperature: will get G different answers with high probability
- resulting in a *group* of G responses to the same q

$$\{(q, o_i) \mid 1 \leq i \leq G\}$$

which are then ranked (to be described).

This is the *Group* part of the GRPO name.

As in most Policy-gradient method

- an advantage is computed for each
- the policy is updated to favor positive advantage responses/disfavor negative advantage responses

In other methods for Preference Data

- each question q provides a *single* opportunity to learn a better policy

GRPO *synthetically* creates G opportunities to learn from each q.

Reward model construction an calibration

Construction

In GRPO the user creates a simple Reward Model

- by writing a function that measures *qualitative* aspects of the response
- typically
    - a sum of several component qualitative measures

To illustrate, suppose we want to post-train a model to "think before answering".

One quality metric is whether the response o_i *obeys the format* of a "thinking" response

```
<think> Thought_1, Thought_2, ... </think> <answer> Answer </answer>
```

The function for this component of the reward can assign higher rewards the closer o_i is to the ideal format

- presence and ordering of tags `<think>`, `</think>`, `<answer>`, `</answer>`
- number of thinking steps
    - not too few or too many

A second very important quality metric: is o_i a correct response to q ?

- an easily evaluated binary measure for problems with verifiable responses (e.g. math problems)
- AI Judge to measure quality of problems with non-verifiable responses

The final Reward sums up the component rewards.

The user can define the relative importance of the components

- weighted sum or different reward scales for each component

Calibration

A key issue with human constructed rewards is *calibration*.

For

- two different classes of problem instance
- or two different humans assigning rewards to instances of the same class
    - or even the same instance

are the two rewards on a comparable scale ?

For example

- Human 1 uses a scale of 0 to 10
- Human 2 uses a scale of 0 to 100

GRPO uses rewards

- in order to *rank* each response o_i in the group of responses for questions q.

Ranks don't have magnitude (just order)

- so ranks across groups are comparable

Human 1 and Human 2, even though they may use different scales

- both wind up with ranks in the range $[1, G]$

This ranking is the source of the *Group Relative* parts of the GRPO name.

Advantage definition

The Group Relative ranking is implemented via the definition of the Advantage function.

In GRPO, the Advantage of a response is defined

- **relative** to its group
- rather than an *absolute* advantage

Given a prompt/input x

- a group $g = \mathrm{group}(\mathrm{x})$ of size G sample responses are collected

For each sample response $y_i \in g$

- there is a corresponding reward $\backslash\mathrm{rewseq}(g, y_i)$

The rewards within group g are normalized, giving the advantage for y_i as

$$A_{g,y_i} = \frac{\backslash\mathrm{rewseq}(g, y_i) - \mu_g}{\sigma_g}$$

where (μ_g, σ_g) are the mean and standard deviation of the rewards withing group g

By subtracting the mean

- responses with below average rewards are penalized
- responses with above average rewards are favored

By normalizing via the standard deviation

- the Advantages of the responses to different queries
- are on a **similar scale**
    - z-score relative to its group

The Surrogate Loss for GRPO

The Surrogate Loss for GRPO is a more complicated version of the Surrogate Loss for PPO.

Recall:

The Surrogate Loss for PPO is

$$
J_{\mathrm{PPO}}(\theta) = \mathbb{E}_\tt \left[
\min{} \left(
r_\tt(\theta) \hat{A}_\tt,\;
\mathrm{clip}\left(r_\tt(\theta), 1 - \epsilon, 1 + \epsilon\right) \hat{\advseq}_\tt
\right)
\right]
$$

The Surrogate Loss for GRPO is

$$J_{\text{GRPO}}(\theta)$$
$$= \mathbb{E}_{(q,a)\sim D,\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{=1}^{|o_i|} \right.$$
$$\left. \min\left(r_{i,}(\theta)\hat{A}_{i,},\ \text{clip}\left(r_{i,}(\theta),1-\epsilon,1+\epsilon\right)\hat{A}_{i,}\right) - \beta \mathrm{D}_{\text{KL}}(\pi_\theta||\pi_{\text{ref}}) \right]$$

Where:

- D is the set of training examples: (q, a) query/answer pairs
- G is the number of sampled outputs per prompt
- $|o_i|$ is the token length of output o_i
- $r_{i,}(\theta) = \frac{\pi_\theta(o_{i,}|q,o_{i,<})}{\pi_{\theta_{\text{old}}}(o_{i,}|q,o_{i,<})}$ is the per-token importance sampling ratio
- $\hat{A}_{i,}$ is the group-relative normalized advantage for token in sample i
- β is the KL penalty coefficient controlling divergence from

We see the familiar

- probability ratio
- clipping

and the addition of a KL divergence term

- in order to further constrain policy changes between epochs

The main difference is in the Expectation

- which is adapted for all members of the group

$$\mathbb{E}_{(q,a)\sim D,\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)}$$

which results in the averaging expressions

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{=1}^{|o_i|}\min$$

- normalization across responses of different length
- averaged over the group

We will have a more detailed discussion of the averaging methodology choice in the section of DAPO.

Relation to the Unified Gradient Formulation

The simplified Policy Gradient Formulation is

$$\nabla_\theta J(\theta) = \mathbb{E}_x \left[\sum_{y \in \text{group(x)}} \nabla_\theta \log \pi_\theta(y|x) A_{\text{group(x),y}} \right]$$

where

- x is an input to the LLM
- $\mathrm{group(x)}$ is a set of LLM outputs, given x as input
    - since the LLM actions are probabilistic
- the advantage $A_{\mathrm{group(x),y}}$
    - of a particular response $y \in \mathrm{group(x)}$
    - is *relative* to other members of $\mathrm{group(x)}$

Discussion

The Surrogate Loss for GRPO and PPO

- are similar in form

But GRPO is a major simplification over PPO

- *eliminates* the need for a Value function
    - using trajectory-level rewards
    - compared to token-level rewards for PPO (derived from the Value function)

GRPO differs from PPO

- adds a KL-constraint to limit the policy update

Group-relative, normalized Advantage

- **Normalization**

The advantage of a response y given input x

- is relative to alternate responses to the same inputs x
    - in units of "number of standard deviations of the group"
- across groups:
    - there is a different standard deviation per group
    - but the response to two different inputs x, x' (and hence groups g, g') are in similar units

Moreover, normalization to mean 0 and unit standard deviation

- reduces variance of gradients
- smoother parameter update

By subtracting a baseline (e.g., the mean)

- policy updates favor/disfavor responses with above/below average rewards

- **Groups**

Having multiple responses per prompt x

- provides multiple updates per prompt, rather than just a single response y

Pseudo code for GRPO

Detailed Surrogate Loss for GRPO

$$
\begin{aligned}
& J_{\mathrm{GRPO}}(\theta) \\
& = \mathbb{E}_{(q,a)\sim D,\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\mathrm{old}}}(\cdot|q)} \Bigg[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{=1}^{|o_i|} \\
& \min\left(r_{i,}(\theta)\hat{A}_{i,},\ \mathrm{clip}\left(r_{i,}(\theta),1-\epsilon,1+\epsilon\right)\hat{A}_{i,}\right) - \beta \mathrm{D}_{\mathrm{KL}}(\pi_\theta||\pi_{\mathrm{ref}}) \Bigg]
\end{aligned}
$$

Where:

- G is the number of sampled outputs per prompt
- $|o_i|$ is the token length of output o_i
- $r_{i,}(\theta) = \frac{\pi_\theta(o_{i,}|q,o_{i,<})}{\pi_{\theta_{\mathrm{old}}}(o_{i,}|q,o_{i,<})}$ is the per-token importance sampling ratio
- $\hat{A}_{i,}$ is the group-relative normalized advantage for token in sample i
- β is the KL penalty coefficient controlling divergence from

```
# GRPO training for LLM
for prompt in training_prompts:
    outputs = [llm.generate(prompt) for _ in range(group_size)]
    advantages = compute_group_relative_advantages(outputs, prompt) # e.g., usi
ng human rank or scoring function

    # Compute loss for all outputs (favor those with high relative advantage)
    losses = []
    for output, advantage in zip(outputs, advantages):
        logprob = llm.logprob(output, prompt)
        losses.append(-logprob * advantage)

    loss = sum(losses) / group_size
    loss.backward()
    optimizer.step()
```

where

```
llm.generate(prompt)
```

is a call to the model, using a prompt `prompt` (e.g., q), to generate a responses (e.g., o_i)

Key Points:

- Multiple candidates sampled per prompt;
- each gets an advantage score
- updates increase likelihood of better completions in the group.

DAPO: An Improved GRPO; Case Study of Real-World

The GRPO model was introduced by DeepSeek, which published the algorithm pseudo-code.

However, as is often the case, the "reference" model (i.e., pseudo-code)

- is *not sufficient* to replicate the performance results quoted in the paper.

There are substantial "engineering details" that are not disclosed.

- The Performance Metrics quoted by DeepSeek
- are 50% higher than those obtained by others trying to replicated the algorithm from the pseudo-code

Details matter !

Responding to this, ByteDance introduced a new model:

- *Decoupled Clip and Dynamic sAmpling Policy Optimization (DAPO)*

in the paper: DAPO: An Open-Source LLM Reinforcement Learning System at Scale (https://arxiv.org/pdf/2503.14476).

DAPO is

- fully Open Source
- that provides refinements to the "reference" GRPO pseudo-code
- sufficient to achieve similar results to the published DeepSeek paper

This was the result of

- experimentation
- error analysis

to diagnose the failures of baseline GRPO and devise remedies.

DAPO introduces 4 key modifications on baseline GRPO

- Changing the threshold for clipping
- *Eliminating* non-informative samples
- Careful redefinition of Token-level Gradient Loss
- Penalizing overly long responses

Evolution of $J_{\mathrm{DAPO}}(\theta)$

DAPO uses a Surrogate Loss that differs from that of GRPO

- in *subtle* ways

To highlight the differences in Surrogate Loss

- we incrementally modify $J_{\mathrm{GRPO}}(\theta)$
- to obtain $J_{\mathrm{DAPO}}(\theta)$

We defer the justification for the modifications until after the final $J_{\mathrm{DAPO}}(\theta)$ has been derived.

Let's start by recalling the Surrogate Loss for GRPO:

$$J_{\text{GRPO}}(\theta)$$

$$= \mathbb{E}_{(q,a)\sim D,\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{G}\sum_{i=1}^{G} \frac{1}{|o_i|} \sum_{=1}^{|o_i|} \right.$$

$$\left. \min\left(r_{i,}(\theta)\hat{A}_{i,},\ \text{clip}\left(r_{i,}(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_{i,}\right) - \beta \mathrm{D}_{\text{KL}}(\pi_\theta||\pi_{\text{ref}}) \right]$$

where

- $\mathcal{D}$ is the training dataset
    - consisting of examples consisting of question/answer pairs
    $$(q, a)$$
- $\{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(\cdot|q)$
    - are the G answers in the group for a questons q

Recall that GRPO

- creates a group of size G answers to q
- by sampling from the LLM with non-zero temperature

Remove KL constraint

First, observe that, relative to GRPO

- DAPO removes the KL-divergence constraint

$$\mathrm{D}_{\mathrm{KL}}\left(\pi_\theta || \pi_{\mathrm{ref}}\right)$$

that limits how far

- new policy π_θ
- can diverge from the *reference* policy π_{ref}.

$$J_{\mathrm{DAPO}^1}(\theta)$$

$$= \mathbb{E}_{(q,a)\sim D,\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\mathrm{old}}}(\cdot|q)}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{=1}^{|o_i|}\min\left(r_{i,}(\theta)\hat{A}_{i,},\ \mathrm{clip}\left(r_{i,}(\theta),1-\epsilon,1+\epsilon\right.\right.\right.$$

giving us the intermediate $J_{\mathrm{DAPO}^1}(\theta)$

Changing the token-level average

GRPO

- First normalizes response advantages to make them independent of response length
    - By dividing by length $|o_i|$ of each response o_i
    - Recall: there is a measure per token of the response
- Then averages sample losses uniformly over group G.

DAPO

- Sums all token losses in the group before normalizing by total tokens in all samples.

giving us the intermediate

$$J_{\text{DAPO}^2}(\theta)$$

$$= \mathbb{E}_{(q,a)\sim\mathcal{D},\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^{G}|o_i|} \sum_{i=1}^{G}\sum_{=1}^{|o_i|} \min \left(r_{i,}(\theta)\hat{A}_{i,},\ \text{clip}\left(r_{i,}(\theta), 1-\epsilon, 1 - \right.\right.\right.$$

This is referred to as the *Token-level Loss* technique in the paper.

Asymmetric Clipping

GRPO's clipping is symmetric with range

$$[1-\epsilon, 1+\epsilon]$$

DAPO adds different clipping values on either side

$$[1-\epsilon_{\text{low}}, 1+\epsilon_{\text{high}}]$$

giving us the intermediate

$$J_{\text{DAPO}^3}(\theta)$$

$$= \mathbb{E}_{(q,a)\sim\mathcal{D},\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^{G}|o_i|} \sum_{i=1}^{G}\sum_{=1}^{|o_i|} \right.$$

$$\left. \min\left(r_{i,}(\theta)\hat{A}_{i,},\ \text{clip}\left(r_{i,}(\theta), 1-\epsilon_{\text{low}}, 1+\epsilon_{\text{high}}\right)\hat{A}_{i,}\right) \right]$$

This is referred to in the paper as the *Clip Higher* technique.

Dynamic sampling

GRPO's expectation is over

- **all** responses within the group for a given question q

DAPO limits the Group composition

- to ensure that **all** intra-group rewards **are not identical**
- expressed as the constraint

$$0 < \left|\{o_i \mid \text{is\_equivalent}(a, o_i)\}\right| < G$$

where

$$\text{is\_equivalent}(a, o_i)$$

is true if response o_i is equivalent to target response a

Thus the expression is a *count* of equivalent responses

- and the group is kept *only if* count is strictly less than G

This gives us the final

$$
\begin{aligned}
& J_{\text{DAPO}}(\theta) \\
& = \mathbb{E}_{(q,a)\sim\mathcal{D},\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^{G}|o_i|} \sum_{i=1}^{G} \sum_{=1}^{|o_i|} \right. \\
& \left. \min \left(r_{i,}(\theta)\hat{A}_{i,},\ \text{clip}\left(r_{i,}(\theta), 1-\epsilon_{\text{low}}, 1+\epsilon_{\text{high}}\right)\hat{A}_{i,} \right) \right],
\end{aligned}
$$

subject to:

$$
0 < \left| \{ o_i \mid \text{is\_equivalent}(a, o_i) \} \right| < G
$$

where a is the (single) response in the training dataset $\mathcal{D}$

- before sampling to generate G sample responses

GRPO vs DAPO Comparison

Feature	GRPO	DAPO				
Loss Aggregation	$ \frac{1}{G}\sum_i \frac{1}{	o_i	} \sum_t $	$ \frac{1} {\sum_i	o_i	} \sum_i \sum_t $
Effect on Long Responses	Weighed less per token (gradient dilution)	Maintains per-token gradient magnitude				
Clipping	Symmetric $(1-\epsilon, 1+\epsilon)$	Asymmetric $(1-\epsilon_{\text{low}}, 1+\epsilon_{\text{high}})$				
Impact	Slower learning on longer tokens	Finer control, better learning on complex outputs				

Aspect	GRPO	DAPO
Value Model Dependency	Removes PPO value model, uses relative group rewards	Same, but more robust and stable optimization
Clipping Mechanism	Basic clip bounds on importance ratios	Asymmetric Clip-Higher for rare, valuable tokens
Sampling	Multi-response batch, potentially redundant	Dynamic Sampling ensures diverse, effective samples
Gradient Weighting	Token-level, gradient diluted in long outputs	Token-Level Gradient Loss corrects signal dilution
Reward for Length	None or truncation-based	Overlong Reward Shaping with soft penalties
Efficiency/Stability	Improved over PPO, but unstable for some scenarios	Superior stability and efficiency; state-of-the-art results

Feature	PPO	GRPO	DAPO
Advantage Source	Value model estimate	Group-relative, reward normalized in batch	Same as GRPO
Clipping	Symmetric $(1-\epsilon, 1+\epsilon)$	Symmetric $(1-\epsilon, 1+\epsilon)$	Asymmetric $(1-\epsilon_{\text{low}}, 1+\epsilon_{\text{high}})$ (Clip-Higher)
Loss Aggregation	Per-sample/token	Per-sample, then averaged over tokens	Per-token, avoids dilution in long outputs
Sampling	Independent samples	Groups of samples per prompt	Dynamic: only informative samples (not all-correct/incorrect)
Reward Shaping	None by default	None by default	Overlong shaping (discourages excessive length)

KL-constraint elimination: why ?

For many RL objectives (e.g., induce long CoT reasoning in the response)

- the *new distribution*
- is necessarily *far* from the reference distribution

So the KL constraint is too limiting for some tasks.

Speculation

DeepSeek initially tried to induce reasoning

- using **only** RL
- **no** SFT step before the RL

As we have seen

- SFT "primes the pump" to make RL more likely to succeed
- by ensuring the initial RL model is able to produce
    - correctly formatted responses

Perhaps the KL-constraint is a vestige of the RL-only attempt

- since the long CoT desired model was far different from the base model

Token-level averaging change: why ?

The GRPO loss aggregation

$$\frac{1}{G}\sum_{i}\frac{1}{|o_i|}\sum_{\dots}$$

creates a loss for sample i in group g

- that is the *average over the tokens* of response o_i

before averaging the sample loss over the G elements of the group.

This introduces a *bias*

- the gradient update of an "important" token in a **long** response o_i
  - important: high gradient
- has less weight in the Gradient of $J_{\text{DAPO}}(\theta)$
- than a token of *equal importance* in a **shorter** response $o_{i'}$

even when the advantages $\hat{A}_i$ and $\hat{A}_{i'}$ are equal.

Reward hacking

This can lead to the undesirable phenomenon known as *Reward Hacking*.

- where the response length is manipulated by the RL feedback
- to change the relative contribution to policy update
- of **important** tokens

RL learns to produce *short **correct** responses*

- with high trajectory (and thus, high per-token) reward
- to amplify the contribution of important tokens

and *long **incorrect** responses*

- with low trajectory (and thus, low per-token) reward
- to dilute the contribution of important tokens
    - adding gibberish or repetitive content in order to increase length

The result is that

- the impact of important tokens
- on updating the policy
- is distorted

DAPO eliminates this bias by changing the aggregation

$$\frac{1}{\sum_i |o_i|} \sum_i \sum_{\ldots}$$

so that

- *all tokens in all responses in a group*
- have the same contribution to the Gradient of $J_{\mathrm{DAPO}}(\theta)$

Reward shaping: Overlong reward shaping length penalty

DAPO also adds a *length penalty* to directly discourage excessively long responses

$$\tilde{R}_i = R_i - \alpha \cdot \mathrm{LengthPenalty}(o_i)$$

where $\mathrm{LengthPenalty}(o_i)$ measures the undesirable properties of response o_i.

The penalty used in the paper is called *Soft Overlong Punishment*

- two length thresholds
- length penalty is phased in between the thresholds

In addition

- long responses are truncated to a maximum length
- the truncated elements of the response are masked in the loss calculation

This is referred to as *Overlong Filtering* in the paper.

Asymmetric Clipping: why ?

Recall the *probability ratio*

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

expresses how much

- the *new probability* $\pi_\theta(a_t|s_t)$ of an action (given a state)
- can differ from
- the *old probability* $\pi_{\theta_{\text{old}}}(a_t|s_t)$
- in *proportional terms*

This ratio is *clipped* in GRPO and DAPO.

In the GRPO loss

- the clipping range is
$$[1-\epsilon, 1+\epsilon]$$

Typically:

$$\epsilon = .20$$

In DAPO

- the clipping range is adjusted to
$$1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}$$

Typically

- $\epsilon_{\text{low}} = \epsilon = .2$
- $\epsilon_{\text{high}} = .28$

Thus, the probability in DAPO is allowed to increase

- more (proportionally) than it is allowed to decrease

The reason for doing so is a simple consequence of

- translating proportional increase to absolute increase

To illustrate, suppose that , at step of the trajectory, there are

- an important action/token $a_{t,\text{low}}$
    - with *low* probability of being chosen by the policy
- a less important action/token $a_{t,\text{high}}$
    - with *high* probability of being chosen by the policy

$$\pi_{\theta_{\text{old}}}(a_{t,\text{low}}|s_t) < \pi_{\theta_{\text{old}}}(a_{t,\text{high}}|s_t)$$

Note

Important is an informal term that refers to the ability to move the policy closer to the optimal one

Here: "low" and "high"

- refer to the policy *probability* and **not** the *importance*

We want to adjust the policy π to

- increase $\pi_{\theta_{\text{old}}}(a_{t,\text{low}}|s_t)$
- decrease $\pi_{\theta_{\text{old}}}(a_{t,\text{high}}|s_t)$

But, because

$$\pi_{\theta_{\text{old}}}(a_{t,\text{low}}|s_t) < \pi_{\theta_{\text{old}}}(a_{t,\text{high}}|s_t)$$

the **absolute increase** in probability

- by multiplying by $(1+\epsilon)$ in the clipping ratio
- for the *low probability* but **more important** token $a_{t,\text{low}}$

is less than the absolute increase

So a proportional increase

- to an initial low probability but important action

has less effect on the Loss (optimization objective)

- than a same proportion increase
- to the initial high probability (but less important) action

Early in learning

- when Exploration (vs Exploitation)
- *could* be high value in moving the policy in the optimal direction
- there is a potential high benefit
- from choosing the *most important* (but low action probability) action

Raising the upper clipping bound has the effect of increasing the entropy of the policy

- more exploration vs exploitation

Dynamic sampling: why ?

If all samples in a group have the same trajectory reward

- the Advantage of each sample in the group
- is mathematically equal to 0

Groups with samples having identical reward are typically

- all samples are correct
- all samples are incorrect

Because the advatange of each sample is 0, these groups

- do not contribute to the Gradient of $J_{\text{DAPO}}(\theta)$

When computing the Gradient of a batch of N questions

- each with G sample responses

the groups with 0 advantage reduce the effective batch size.

This results in such batches having *noisier* gradient updates than unaffected batches.

Moreover the learning signal

- is stronger
- when there is a *contrast* between samples
    - high reward vs. low reward
- whether within a group or within a batch

Dynamic sampling promotes contrastive groups.

On the topic of *contrastive examples*

- without the initial SFT of RFT
- when the behavior to learn via RL is very different than the behavior of the base LLM
- all examples and samples are likely to have the same low reward
    - because of incorrect formatting or logic

So the initial SFT will hopefully create some high reward examples

- by "moving the distribution" of RL training examples
- closer to the ultimate RL goal
- than the distribution of examples from the base (non-SFT tuned) model

Contribution of each technique to the improvement of DAPO vs GRPO

Technique	Description	Commentary	Accuracy Improvement (AIME 2024 avg@32)
Naive GRPO	Baseline group relative policy optimization without enhancements	Starting point with relatively low accuracy	30
Overlong Filtering	Filters out truncated (overlong) samples from training loss	Reduces reward noise caused by forced truncation, stabilizes training	36
Clip-Higher	Decouples lower and upper clipping range to allow higher increase for low-probability tokens	Enhances policy entropy and exploration, avoids early collapse of exploration	38
Soft Overlong Punishment	Length-aware penalty on excessively long responses	Prevents reward noise from overly penalizing valid but long reasoning chains	41
Token-Level Loss	Aggregates loss over tokens normalized by total token count (not per sample average)	Improves training stability and healthier growth in output length	42
Dynamic Sampling	Oversamples and filters batches to keep effective gradient signals by excluding zero-advantage samples	Significantly improves training stability and performance speed	50

Pseudo code for DAPO

Detailed Surrogate Loss for DAPO

$$J_{\mathrm{DAPO}}(\theta)$$

$$= \mathbb{E}_{(q,a),\{o_i\}\sim\pi_{\theta_{\mathrm{old}}}} \left[\frac{1}{\sum_{i=1}^{G}|o_i|} \sum_{i=1}^{G}\sum_{=1}^{|o_i|} \min \left(r_{i,}(\theta)\hat{A}_{i,},\ \mathrm{clip}(r_{i,}(\theta), 1-\epsilon_{\mathrm{low}}, 1+\epsilon_{\mathrm{high}} \right.\right.$$

```
# Given:
# batch_size = N
# num_samples = K
# sequence_lengths = [T_gi for each response i in group g]
# importance_scores = array of same shape as tokens, default = 1

for group_id in range(N):
    for sample_id in range(K):
        T = sequence_lengths[group_id][sample_id]
        importance_sum = 0
        # Compute sum of importance scores in sample
        for t in range(T):
            importance_sum += importance_scores[group_id][sample_id][t]
        # Calculate alpha for each token
        for t in range(T):
            alpha = importance_scores[group_id][sample_id][t] / importance_sum
            # Compute token-wise policy gradient component:
            grad = alpha * w[group_id][sample_id] \
                   * clip(r[group_id][sample_id][t], 1-eps_low, 1+eps_high) \
                   * advantage[group_id][sample_id][t]
```

References for GRPO to DAPO

- DAPO: An Open-Source LLM Reinforcement Learning System at Scale (arXiv) (https://arxiv.org/abs/2503.14476)
- Mathematics of DAPO, PPO, and GRPO (SSRN) (https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5205449)
- From GRPO to DAPO and GSPO: What, Why, and How (Hugging Face blog) (https://huggingface.co/blog/NormalUhr/grpo-to-dapo-and-gspo)
- The Evolution of GRPO: DAPO (Towards AI) (https://towardsai.net/p/l/the-evolution-of-grpo-dapo)

Comparison of methods for Preference Data

Aspect	PPO (Proximal Policy Optimization)	DPO (Direct Preference Optimization)	GRPO (Group Relative Policy Optimization)
Stability	Moderate stability, uses clipped objective to limit policy updates and prevent divergence. Can still be sensitive to reward noise and hyperparameters.	High stability due to supervised-learning style objective on preference pairs. Does not rely on policy gradient RL steps.	Higher stability than PPO due to normalized group rewards reducing gradient noise; does not require a value function critic which reduces instability.
Variance	High variance in gradient estimates caused by sparse rewards and stochastic policy sampling. Requires variance reduction techniques (e.g., baseline/critic).	Low variance because gradients come from direct supervised preference comparisons without sampling or policy gradients.	Moderate variance—variance is reduced by reward normalization within groups but still involves sampling multiple outputs, so more variance than DPO but less than PPO.
Sample Efficiency	Moderate to low—needs many environment interactions/samples due to sparse reward signal and on-policy updates. Sampling multiple sequences per prompt increases cost.	Very high—trains directly on labeled preference pairs with no complex sampling or reward modeling.	Higher than PPO—requires multiple samples per prompt for group comparison but gains efficiency from relative advantage normalization and critic-free updates.

In [3]:
```
print("Done")
```

```
Done
```